

ACADEMIA DE STUDII ECONOMICE DIN BUCUREȘTI

Facultatea de Cibernetică, Statistică și Informatică Economică

Specializare: Informatică Economică



PROIECT PACHETE SOFTWARE

Analiza platformei de streaming Spotify utilizând Python și SAS

Coordonator științific

Conferențiar univ. dr. ANDREESCU Anca Ioana

Studenti

BODEA ȘTEFANIA-DENISA

BOGDAN MIHAELA

BUCUREȘTI 2026

1. Definirea problemei.....	3
2. Informații necesare pentru rezolvare.....	3
3. Metode de calcul, algoritmi, formule de calcul utilizate și prezentarea rezultatelor.....	4
3.1. Componenta Python.....	4
3.1.1. Utilizarea metodelor Streamlit pentru afișare și interfață.....	5
3.1.2. Încărcarea și afișarea datasetului.....	6
3.1.3. Tratarea valorilor lipsă.....	7
3.1.4. Tratarea valorilor extreme.....	8
3.1.5. Codificarea datelor categoricale.....	9
3.1.6. Scalarea datelor numerice.....	9
3.1.7. Prelucrări statistice, grupare și agregare în pandas.....	10
3.1.8. Utilizarea funcțiilor de grup.....	12
3.1.9. Clustering K-Means cu scikit-learn.....	12
3.1.10. Regresie multiplă cu statsmodels.....	13
3.1.11. Clasificare prin regresie logistică.....	15
3.1.12. Vizualizări grafice și analize avansate.....	16
3.2 Componenta SAS.....	19
3.2.1. Crearea unui set de date SAS din fișiere externe.....	19
3.2.2. Crearea și folosirea de formate definite de utilizator.....	19
3.2.3. Procesarea iterativă și condițională a datelor.....	20
3.2.4. Subseturi.....	21
3.2.5. Funcții SAS.....	22
3.2.6. Combinarea seturilor de date (MERGE + SQL).....	23
3.2.7. Arrays.....	24
3.2.8. Proceduri statistice.....	25
3.2.9. SAS ML.....	26
3.2.10. Grafice.....	28
4. Interpretarea economica a rezultatelor.....	30

1. Definirea problemei

Proiectul analizează un set de date Spotify cu informații despre piese, artiști, genuri și caracteristici audio precum energie, dansabilitate și tempo. Scopul este de a transforma aceste date brute în informații utile pentru înțelegerea catalogului muzical și identificarea pieselor cu potențial.

Analiza este realizată în două componente: Python, prin care am construit o aplicație interactivă în Streamlit, și SAS, folosit pentru prelucrări statistice, raportări și grafice. Împreună, cele două pachete ne ajută să răspundem la întrebări precum: ce genuri sunt cele mai populare, ce caracteristici audio definesc o piesă de succes și care sunt direcțiile posibile de promovare.

2. Informații necesare pentru rezolvare

Setul de date inițial

Setul de date conține informații despre piese muzicale disponibile pe platforma Spotify, fiind structurat sub formă de tabel unde fiecare rând corespunde unei piese. Coloanele conțin atât informații de identificare (titlu, artist, gen), cât și caracteristici audio măsurate algoritmic de platformă. Setul de date conține 5.000 de observații (piese) și 22 de coloane (variabile) și a fost preluat de pe Kaggle.

Variabilele din setul de date

- **track_id**: identificatorul unic al piesei în sistem;
- **track_name**: titlul piesei;
- **artist_name**: numele artistului sau trupei;
- **genre**: genul muzical al piesei (ex: pop, rock, hip-hop, jazz etc.);
- **year**: anul lansării piesei;
- **popularity**: scorul de popularitate al piesei pe Spotify, exprimat pe o scală de la 0 la 100;
- **duration_ms**: durata piesei exprimată în milisecunde;
- **explicit**: indică dacă piesa conține conținut explicit (1 = da, 0 = nu);
- **danceability**: măsoară cât de potrivită este piesa pentru dans, pe o scală de la 0 la 1;

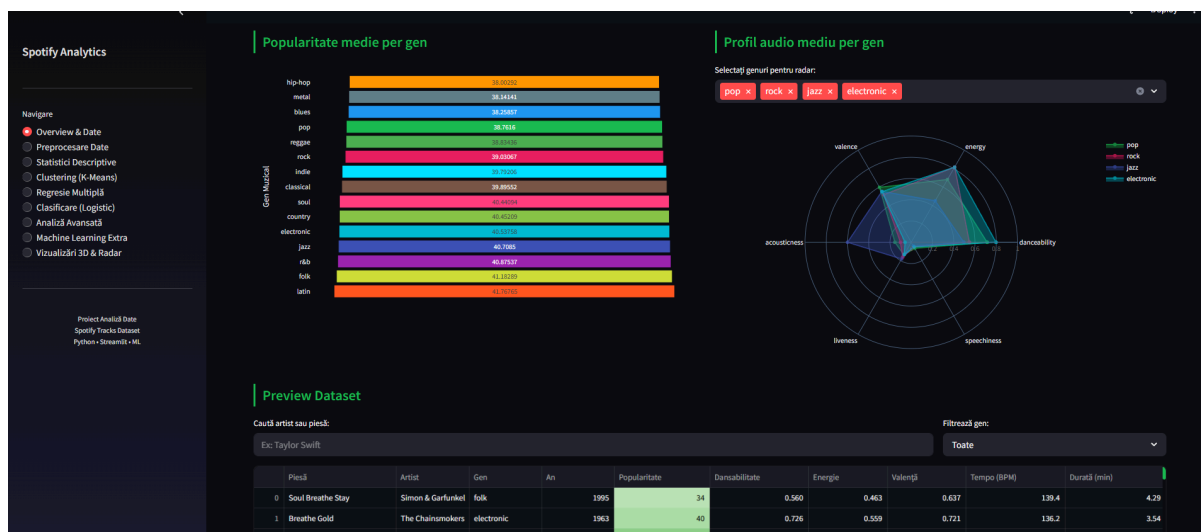
- **energy**: măsoară intensitatea și activitatea piesei, pe o scală de la 0 la 1;
- **key**: tonalitatea piesei exprimată numeric (0 = Do, 1 = Do#, etc.);
- **key_name**: tonalitatea piesei exprimată ca literă (ex: C, G, F#);
- **loudness**: volumul mediu al piesei, exprimat în decibeli (dB);
- **mode**: 1 pentru Major, 0 pentru Minor;
- **speechiness**: măsoară prezența cuvintelor rostite în piesă, pe o scală de la 0 la 1;
- **acousticness**: măsoară cât de acustică este piesa, pe o scală de la 0 la 1;
- **instrumentalness**: măsoară probabilitatea că piesa nu conține voce, pe o scală de la 0 la 1;
- **liveness**: măsoară probabilitatea că piesa a fost înregistrată live, pe o scală de la 0 la 1;
- **valence**: măsoară pozitivitatea muzicală a piesei; valori mari indică piese vesele, valori mici indică piese triste, pe o scală de la 0 la 1.
- **tempo**: tempo-ul piesei exprimat în bătăi pe minut (BPM);
- **time_signature**: măsura muzicală a piesei (ex: 3, 4);
- **duration_min**: durata piesei exprimată în minute.

3. Metode de calcul, algoritmi, formule de calcul utilizate și prezentarea rezultatelor

3.1. Componenta Python

Pentru realizarea interfeței aplicației s-a utilizat biblioteca **Streamlit**, care permite construirea unui dashboard interactiv direct în Python. În proiect, aplicația este pornită din fișierul `app.py`, unde se configurează pagina, meniul lateral și legătura către secțiunile principale ale dashboardului.

Meniul lateral permite navigarea între paginile aplicației: **Overview & Date**, **Preprocesare Date**, **Statistici Descriptive**, **Clustering (K-Means)**, **Regresie Multiplă**, **Clasificare (Logistic)**, **Analiză Avansată**, **Machine Learning Extra** și **Vizualizări 3D & Radar**. Fiecare pagină tratează o etapă diferită a analizei datelor.



3.1.1. Utilizarea metodelor Streamlit pentru afișare și interfață

În aplicație sunt folosite metode Streamlit precum `st.title()`, `st.markdown()`, `st.dataframe()`, `st.metric()`, `st.columns()`, `st.sidebar`, `st.selectbox()` și `st.plotly_chart()`. Acestea permit afișarea titlurilor, textelor explicative, tabelelor, cardurilor KPI, filtrelor interactive și graficelor.

`st.markdown` afișează titluri și text formatat direct în interfață.

`st.columns(5)` împarte pagina în 5 coloane egale, în care sunt afișate cardurile KPI, fiind câte un indicator cheie pentru fiecare coloană. Valorile sunt calculate direct din DataFrame și afișate cu HTML personalizat prin `unsafe_allow_html=True`.

```

st.markdown("# Spotify Tracks – Prezentare Generală")
st.markdown(
    "Analiză completă a unui dataset cu **5.000 de piese Spotify** cuprinzând "
    "15 genuri muzicale, caracteristici audio și metadate de popularitate."
)

# KPI Cards
c1, c2, c3, c4, c5 = st.columns(5)
kpis = [
    (c1, len(df), "Piese totale"),
    (c2, df["genre"].nunique(), "Genuri"),
    (c3, df["artist_name"].nunique(), "Artiști"),
    (c4, f"{df['year'].min()}–{df['year'].max()}", "Interval ani"),
    (c5, f"{df['popularity'].mean():.1f}", "Popularitate medie"),
]
for col, val, label in kpis:
    col.markdown(f"""
    <div class='metric-card'>
        <div class='metric-value'>{val}</div>
        <div class='metric-label'>{label}</div>
    </div>
    """, unsafe_allow_html=True)

```



Cardurile KPI oferă o imagine de ansamblu rapidă asupra celor 5.000 de piese din 15 genuri muzicale, lansate pe o perioadă de câteva decenii, cu o popularitate medie care reflectă că majoritatea pieselor din catalog sunt de nișă, nu virale.

3.1.2. Încărcarea și afișarea datasetului

Prima etapă a aplicației constă în încărcarea datasetului `spotify_tracks.csv`. Încărcarea datelor se realizează în fișierul `data_loader.py`, prin funcția **`load_data()`**.

Pentru citirea fișierului se folosește biblioteca **`pandas`**, prin instrucțiunea `pd.read_csv("spotify_tracks.csv")`. Datasetul este transformat într-un `DataFrame`, care este folosit ulterior în toate secțiunile aplicației. Pentru eficiență, funcția utilizează **`@st.cache_data`**, astfel încât datele să nu fie recitite la fiecare schimbare de pagină sau filtrare.

```
@st.cache_data
def load_data() -> pd.DataFrame:
    ⚡ try:
        df = pd.read_csv("spotify_tracks.csv")
    except FileNotFoundError:
        st.error("❌ Fișierul `spotify_tracks.csv` nu a fost găsit. Rulați")
        st.stop()
    return df
```

`pd.read_csv` încarcă fișierul CSV în memorie ca `DataFrame` `pandas`, iar **`st.session_state`** îl salvează în sesiunea curentă astfel încât să nu fie recitit la fiecare interacțiune cu interfața.

`df.copy()` creează o copie de lucru pentru a păstra datele originale intacte. **`st.dataframe`** afișează tabelul interactiv în interfață cu coloanele redenumite în română, valorile formate la numărul corespunzător de zecimale și un gradient verde pe coloana de popularitate care face vizuală diferența dintre piese populare și cele de nișă.

```
display_cols = ["track_name", "artist_name", "genre", "year", "popularity",
                "danceability", "energy", "valence", "tempo", "duration_min"]
st.dataframe(
    df_view[display_cols].rename(columns={
        "track_name": "Piesă", "artist_name": "Artist", "genre": "Gen",
        "year": "An", "popularity": "Popularitate", "danceability": "Dansabilitate",
        "energy": "Energie", "valence": "Valență", "tempo": "Tempo (BPM)",
        "duration_min": "Durată (min)"
    }).style.format({
        "Popularitate": "{:.0f}",
        "Dansabilitate": "{:.3f}", "Energie": "{:.3f}",
        "Valență": "{:.3f}", "Tempo (BPM)": "{:.1f}",
        "Durată (min)": "{:.2f}",
    }).background_gradient(subset=["Popularitate"], cmap="Greens"),
    use_container_width=True,
    height=350,
)
st.caption(f" {len(df_view):,} piese afișate din {len(df):,} total")
```

Preview Dataset

Caută artist sau piesă: Filtrează gen:

	Piesă	Artist	Gen	An	Popularitate	Dansabilitate	Energie	Valență	Tempo (BPM)	Durată (min)
27	Run Touch Today	Green Day	rock	1965	29	None	1.000	0.692	94.2	3.31
28	Dreams	Foo Fighters	rock	1979	40	0.667	0.805	0.351	123.6	3.45
29	Lost Moon Found	Arcade Fire	indie	1991	44	0.655	0.739	0.385	106.0	3.49
30	Moon Gold Found	Modest Mouse	indie	1982	38	0.589	0.501	0.584	145.6	4.01
31	Sun	Ludwig van Beethoven	classical	1988	44	0.305	0.553	0.473	82.0	3.03
32	Black Night Rise	Claude Debussy	classical	1968	43	0.088	0.217	0.263	99.5	7.90
33	Cry Rise Free	Kanye West	hip-hop	1964	35	0.898	0.734	0.526	93.7	3.74
34	Always Dark Wild	The Weeknd	r&b	2015	74	None	0.508	0.521	100.8	4.05
35	Mauha Sihar	Pantera	metal	1975	37	0.635	0.850	0.304	151.1	5.68

5,000 piese afișate din 5,000 total

Structura completă a dataset-ului

Coloană	Tip date	Valori lipsă	Valori unice	Exemplu
mode	int64	0	2	1
speechiness	float64	0	4819	0.0715842563101478
acousticness	float64	0	4568	0.6078523925497995
instrumentalness	float64	0	3848	0.0312690434440655
liveness	float64	0	4854	0.2013947783322327
valence	float64	0	4957	0.6365839368581246

Aplicația permite filtrarea datelor după numele piesei, artist sau gen muzical, ceea ce face explorarea datasetului mai ușoară. De asemenea, prin secțiunea **Structura completă a dataset-ului**, utilizatorul poate vedea tipurile de date, valorile lipsă, valorile unice și exemple pentru fiecare coloană.

3.1.3. Tratarea valorilor lipsă

Această etapă este necesară deoarece datasetul poate conține informații incomplete despre anumite piese, iar aceste lipsuri pot afecta analizele statistice și modelele de machine learning. Pentru identificarea valorilor lipsă se folosesc funcții din biblioteca **pandas**, precum `isnull()`, `isna()` și `sum()`. După tratare, datele curate sunt utilizate în paginile de statistici, clustering, regresie și clasificare.

`df.isna().sum()` numără valorile lipsă din fiecare coloană și le afișează într-un tabel în coloana din stânga, înainte de tratare.

`fillna(median())` înlocuiește fiecare valoare lipsă cu mediana coloanei respective. Se folosește mediana în loc de medie pentru că este mai robustă la valorile extreme.

```
st.subheader("1. Tratarea valorilor lipsă și extreme")
col1, col2 = st.columns(2)

with col1:
    st.write("Valori lipsă inițiale:")
    st.dataframe(df.isna().sum()[df.isna().sum() > 0])

# Tratare valori lipsă
df['danceability'] = df['danceability'].fillna(df['danceability'].median())
df['energy'] = df['energy'].fillna(df['energy'].median())
df['tempo'] = df['tempo'].fillna(df['tempo'].median())
df['popularity'] = df['popularity'].fillna(df['popularity'].median())

with col2:
    st.write("După tratare (imputare cu mediană):")
    st.dataframe(df.isna().sum()[df.isna().sum() > 0])
    st.success("Valorile lipsă și extreme au fost tratate!")
```

3.1.4. Tratarea valorilor extreme

În pagina **Preprocesare Date**, aplicația tratează și valorile extreme, adică acele observații care se abat mult față de restul datelor. Valorile extreme sunt corectate prin reguli aplicate asupra coloanelor numerice. Această etapă este importantă deoarece outlierii pot influența mediile, graficele și modelele de machine learning. Prin tratarea lor, datasetul devine mai stabil și mai potrivit pentru analize ulterioare, precum clusteringul sau regresia.

df.loc[conditie, coloana] selectează doar rândurile care îndeplinesc condiția specificată și înlocuiește valorile respective.

Pentru **tempo**, orice valoare peste 200 BPM este considerată eronată și înlocuită cu mediana coloanei.

Pentru **popularity**, valorile peste 100 sunt imposibile din punct de vedere al scalei platformei Spotify și sunt taiate la valoarea maximă validă de 100.

Pentru **loudness**, valorile sub -60 dB reprezintă piese practic inaudibile și sunt înlocuite cu mediana

```
# Tratare valori extreme (ex: tempo > 200, popularity > 100)
df.loc[df['tempo'] > 200, 'tempo'] = df['tempo'].median()
df.loc[df['popularity'] > 100, 'popularity'] = 100
df.loc[df['loudness'] < -60, 'loudness'] = df['loudness'].median()
```

Preprocesare Date

În această secțiune tratăm valorile lipsă, codificăm datele categorice și scalăm variabilele numerice.

1. Tratarea valorilor lipsă și extreme

Valori lipsă inițiale:

	0
popularity	45
danceability	162
energy	112
tempo	99

După tratare (imputare cu mediană):

	0
popularity	empty
danceability	empty
energy	empty
tempo	empty

Valorile lipsă și extreme au fost tratate!

3.1.5. Codificarea datelor categoricale

În aplicație, codificarea datelor categoricale este realizată în pagina **Preprocesare Date**. Această etapă este necesară deoarece datasetul conține variabile textuale, precum **genre**, care nu pot fi folosite direct în anumite modele de machine learning.

LabelEncoder din **scikit-learn** transformă valorile textuale ale coloanei **genre** în numere întregi atribuindu-i fiecărui gen unic un număr de la 0 la 14 (pentru cele 15 genuri).

fit_transform învață corespondența dintre etichete și numere și o aplică simultan.

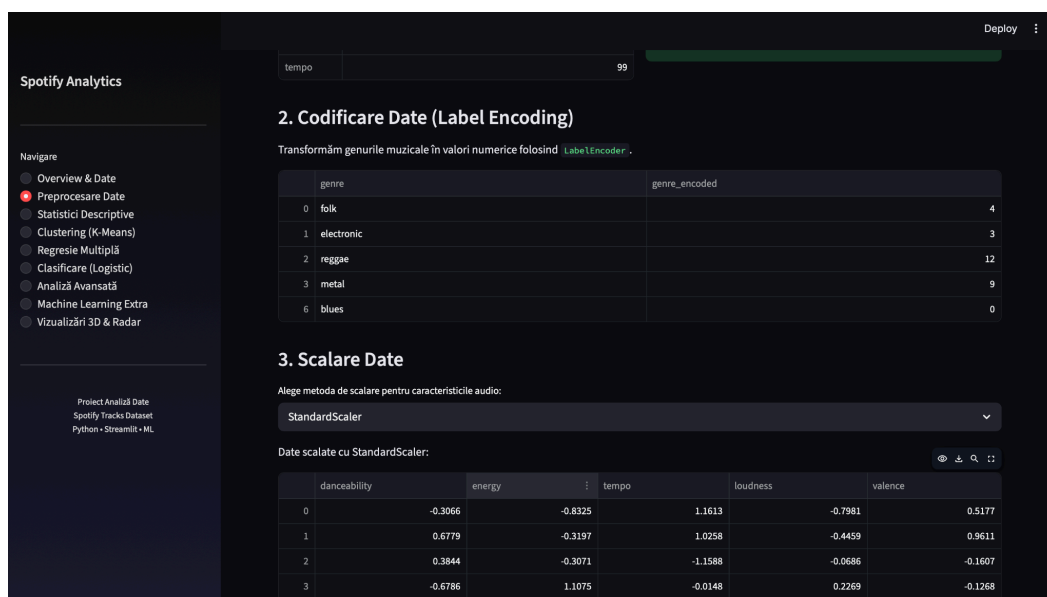
drop_duplicates() păstrează doar câte un rând per gen pentru a afișa un tabel de corespondență curat, fără repetări.

```
st.subheader("2. Codificare Date (Label Encoding)")
st.write("Transformăm genurile muzicale în valori numerice folosind `LabelEncoder`.")
le = LabelEncoder()
df['genre_encoded'] = le.fit_transform(df['genre'])

st.dataframe(df[['genre', 'genre_encoded']].drop_duplicates().head(5))
```

Aplicația afișează un tabel cu exemple de genuri și codurile asociate, pentru a arăta clar cum s-a realizat transformarea. Această prelucrare permite folosirea informației despre genul muzical în analize numerice și modele predictive.

Din punct de vedere economic, codificarea genurilor ajută la compararea segmentelor muzicale și la integrarea acestora în modele care pot evidenția genurile cu potențial de popularitate sau promovare.



Spotify Analytics

Navigate

- Overview & Date
- Preprocesare Date**
- Statistici Descriptive
- Clustering (K-Means)
- Regresie Multiplă
- Clasificare (Logistic)
- Analiză Avansată
- Machine Learning Extra
- Vizualizări 3D & Radar

Project Available Data
Spotify Tracks Dataset
Python • Streamlit • ML

tempo 99

2. Codificare Date (Label Encoding)

Transformăm genurile muzicale în valori numerice folosind `LabelEncoder`.

genre	genre_encoded
0 folk	4
1 electronic	3
2 reggae	12
3 metal	9
6 blues	0

3. Scalare Date

Alege metoda de scalare pentru caracteristicile audio:

StandardScaler

Date scalate cu StandardScaler:

	danceability	energy	tempo	loudness	valence
0	-0.3066	-0.8325	1.1613	-0.7981	0.5177
1	0.6779	-0.3197	1.0258	-0.4459	0.9611
2	0.3844	-0.3071	-1.1588	-0.0686	-0.1607
3	-0.6786	1.1075	-0.0148	0.2269	-0.1268

3.1.6. Scalarea datelor numerice

Scalarea datelor numerice este realizată tot în pagina **Preprocesare Date**. Această etapă este importantă deoarece variabilele din dataset au scări diferite: de exemplu, tempo are valori mult mai mari decât danceability, energy sau valence.

În aplicație, utilizatorul poate alege metoda de scalare printr-un selector: **StandardScaler** sau **MinMaxScaler**. Sunt scalate caracteristici audio precum danceability, energy, tempo, loudness și valence, deoarece acestea sunt folosite ulterior în analize și modele de machine learning.

Prin scalare, variabilele sunt aduse la o scară comparabilă, astfel încât algoritmi precum K-Means sau regresia logistică să nu fie influențați excesiv de variabilele cu valori numerice mai mari.

```
st.subheader("3. Scalare Date")
scaler_type = st.selectbox("Alege metoda de scalare pentru caracteristicile audio:", ["StandardScaler", "MinMaxScaler"])

features_to_scale = ['danceability', 'energy', 'tempo', 'loudness', 'valence']

if scaler_type == "StandardScaler":
    scaler = StandardScaler()
else:
    scaler = MinMaxScaler()

df[features_to_scale] = scaler.fit_transform(df[features_to_scale])

st.write(f"Date scalate cu {scaler_type}:")
st.dataframe(df[features_to_scale].head())
```

Variabilele din dataset au scale foarte diferite: tempo are valori între 60 și 200 BPM, în timp ce danceability este între 0 și 1. Fără scalare, algoritmi bazați pe distanță precum K-Means ar fi dominați de tempo și ar ignora practic celelalte caracteristici. Scalarea aduce toate variabilele pe același plan și asigură că fiecare contribuie echilibrat la analiză.

Spotify Analytics

Navigare

- Overview & Date
- Preprocesare Date
- Statistici Descriptive
- Clustering (K-Means)
- Regresie Multiplă
- Clasificare (Logistic)
- Analiză Avansată
- Machine Learning Extra
- Vizualizări 3D & Radar

Proiect Analiză Date
Spotify Tracks Dataset
Python • Streamlit • ML

2	reggae	12
3	metal	9
6	blues	0

3. Scalare Date

Alege metoda de scalare pentru caracteristicile audio:

StandardScaler

Date scalate cu StandardScaler:

	danceability	energy	tempo	loudness	valence	
0	-0.3066	-0.8325	1.1613	-0.7981	0.5177	
1	0.6779	-0.3197	1.0258	-0.4459	0.9611	
2	0.3844	-0.3071	-1.1588	-0.0686	-0.1607	
3	-0.6786	1.1075	-0.0148	0.2269	-0.1268	
4	-0.711	0.9468	0.9929	0.5486	-0.7314	

Datele au fost preprocesate și salvate în sesiune pentru analizele următoare!

3.1.7. Prelucrări statistice, grupare și agregare în pandas

În aplicație, prelucrările statistice sunt realizate în pagina **Statistici Descriptive & Grupări**. Această secțiune folosește datasetul preprocesat anterior și calculează indicatori importanți pentru variabilele numerice, precum media, mediana, minimul, maximul și abaterea standard.

Pentru aceste calcule se utilizează biblioteca **pandas**, în special funcții precum `describe()`, `groupby()` și `agg()`. Aplicația grupează datele după coloana `genre` și calculează indicatori precum popularitatea medie, energia medie, dansabilitatea medie, tempo-ul mediu, durata medie și numărul de piese pentru fiecare gen muzical.

```
numeric_cols = ["popularity", "danceability", "energy", "valence",  
                "tempo", "acousticness", "instrumentalness",  
                "speechiness", "liveness", "loudness", "duration_min"]  
desc = df[numeric_cols].describe().round(4)  
st.dataframe(desc.style.background_gradient(cmap="Greens"), use_container_width=True)
```

```
grouped = df.groupby("genre").agg(  
    Popularitate_Medie = ("popularity", "mean"),  
    Popularitate_Mediana = ("popularity", "median"),  
    Energie_Medie = ("energy", "mean"),  
    Dansabilitate_Medie = ("danceability", "mean"),  
    Valenta_Medie = ("valence", "mean"),  
    Tempo_Mediu = ("tempo", "mean"),  
    Durata_Medie_min = ("duration_min", "mean"),  
    Nr_Piese = ("track_id", "count"),  
    Nr_Artisti_Unici = ("artist_name", "nunique"),  
    Explicit_Procent = ("explicit", "mean"),  
) .reset_index().sort_values("Popularitate_Medie", ascending=False).round(3)  
  
st.dataframe(grouped, use_container_width=True)
```

Rezultatele sunt afișate sub formă de tabele și grafice interactive. De exemplu, aplicația poate evidenția genurile cu popularitate medie mai mare sau genurile care au piese mai energice și mai dansabile.

Spotify Analytics

Navigare

Overview & Date

Preprocesare Date

Statistici Descriptive

Clustering (K-Means)

Regresie Multiplă

Clasificare (Logistic)

Analiză Avansată

Machine Learning Extra

Vizualizări 3D & Radar

Proiect Analiză Date

Spotify Tracks Dataset

Python • Streamlit • ML

Deploy

Statistici Descriptive & Grupări

1. Statistici descriptive generale (pandas describe)

	popularity	danceability	energy	valence	tempo	acousticness	instrumentalness	speechiness	liveness	loudness	dur
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	50
mean	39.772200	-0.000000	-0.000000	0.000000	-0.000000	0.322200	0.165300	0.069300	0.150100	-0.000000	
std	20.691900	1.000100	1.000100	1.000100	1.000100	0.268200	0.258900	0.060300	0.080700	1.000100	
min	0.000000	-3.616400	-3.325000	-2.823600	-2.982900	0.000000	0.000000	0.000000	0.000000	-7.368800	
25%	25.000000	-0.666500	-0.644400	-0.697300	-0.671800	0.102300	0.005200	0.032600	0.093100	-0.401700	
50%	39.000000	0.110600	0.003800	0.017200	-0.014800	0.253600	0.049000	0.054800	0.145800	0.183900	
75%	54.000000	0.762300	0.730600	0.707400	0.646000	0.482400	0.170700	0.086200	0.202600	0.643600	
max	100.000000	2.298000	2.053700	2.425100	3.683900	1.000000	1.000000	0.471800	0.476100	1.668000	

Rândurile arată: count, mean, std, min, 25%, 50%, 75%, max pentru fiecare coloană numerică.

2. Agregări cu pandas groupby

Calculăm mai multe statistici agregate pentru fiecare gen muzical, utilizând funcțiile de grup: `mean`, `median`, `std`, `count`, `min`, `max`.

	genre	Popularitate_Medie	Popularitate_Mediana	Energie_Medie	Dansabilitate_Medie	Valenta_Medie	Tempo_Mediu	Durata_Medie_min	Nr_P
8	latin	41.76	42.5	0.55	0.939	0.857	-0.238		3.612
4	folk	41.176	41	-0.718	-0.558	-0.059	-0.056		3.877

Spotify Analytics

Navigare

Overview & Date

Preprocesare Date

Statistici Descriptive

Clustering (K-Means)

Regresie Multiplă

Clasificare (Logistic)

Analiză Avansată

Machine Learning Extra

Vizualizări 3D & Radar

Proiect Analiză Date

Spotify Tracks Dataset

Python • Streamlit • ML

Deploy

2. Agregări cu pandas groupby

Calculăm mai multe statistici agregate pentru fiecare gen muzical, utilizând funcțiile de grup: `mean`, `median`, `std`, `count`, `min`, `max`.

	genre	Popularitate_Medie	Popularitate_Mediana	Energie_Medie	Dansabilitate_Medie	Valenta_Medie	Tempo_Mediu	Durata_Medie_min	Nr_P
8	latin	41.76	42.5	0.55	0.939	0.857	-0.238		3.612
4	folk	41.176	41	-0.718	-0.558	-0.059	-0.056		3.877
11	r&b	40.864	40	-0.192	0.543	-0.119	-0.477		3.741
7	jazz	40.672	40	-0.888	-0.666	0.079	0.117		4.923
3	electronic	40.458	39	1.046	1.064	0.062	0.659		4.118
14	soul	40.388	40	-0.047	0.196	0.216	-0.291		3.706
2	country	40.382	39	-0.012	-0.017	0.406	0.328		3.572
1	classical	39.875	39	-1.758	-1.781	-0.713	-0.554		6.966
6	indie	39.729	39.5	-0.093	-0.348	-0.331	0.138		3.967
13	rock	39.031	39	1.038	-0.364	-0.114	0.843		3.887

Popularitate Medie per Gen

Energie vs. Popularitate (per Gen)

>>

12	reggae	38.836	38	-0.313	0.606	0.714	-1.278	3.915	329	10	0.073
----	--------	--------	----	--------	-------	-------	--------	-------	-----	----	-------

Deploy

Popularitate Medie per Gen

Energie vs. Popularitate (per Gen)

3. Funcția transform — Popularitate relativă față de media genului

Folosim `groupby().transform('mean')` pentru a calcula cât de populară este fiecare piesă față de media genului ei.

	track_name	artist_name	genre	popularity	pop_medie_gen	pop_relativa
3132	Dreams Broken	Kendrick Lamar	hip-hop	100	38.0058	61.99
3947	Silver Dark Soul	Ke\$ha	blues	100	38.2677	61.73
628	Black	Billie Eilish	pop	100	38.7219	61.28

14

Statisticile descriptive arată că popularitatea medie din dataset este în jurul valorii de 40, cu o deviație standard mare, semn că există o diferență semnificativă între piesele de nișă și cele virale.

Agregarea pe genuri relevă că latin și folk au cea mai mare popularitate medie, în timp ce genuri precum blues și hip-hop se situează la coada clasamentului, informație utilă pentru un label care vrea să prioritizeze investițiile în anumite genuri.

3.1.8. Utilizarea funcțiilor de grup

Funcțiile de grup sunt utilizate în aplicație pentru a analiza datasetul pe categorii, în special după genul muzical. Prin gruparea datelor, aplicația poate calcula indicatori separați pentru fiecare gen, ceea ce permite o comparație mai clară între categorii.

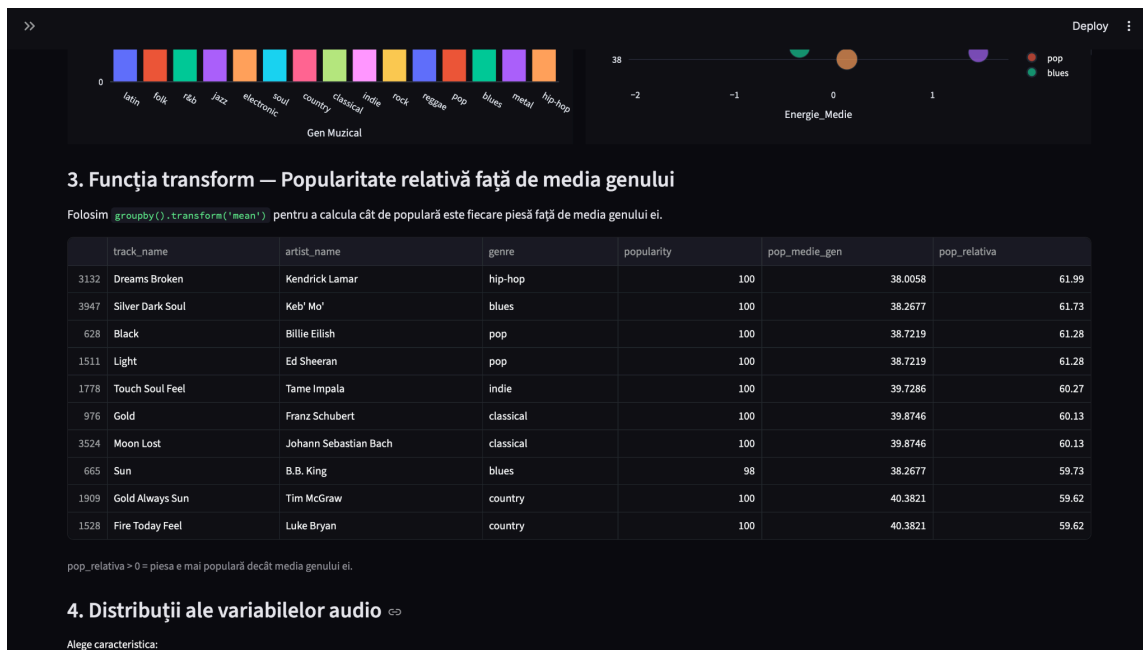
groupby("genre")["popularity"].transform("mean") este diferit față de **groupby().agg()** deoarece în loc să returneze un tabel rezumat cu câte un rând per gen, transform returnează o coloană de aceeași lungime ca DataFrame-ul original, unde fiecare piesă primește media genului său. Astfel, pentru fiecare piesă se poate calcula **pop_relativa**, diferența dintre popularitatea piesei și media genului din care face parte.

sort_values ordonează piesele descrescător după această diferență, afișând top 10 piese care depășesc cel mai mult media genului lor.

```
st.subheader("3. Funcția transform – Popularitate relativă față de media genului")
st.write("Folosim `groupby().transform('mean')` pentru a calcula cât de populară "
        "este fiecare piesă față de media genului ei.")
df["pop_medie_gen"] = df.groupby("genre")["popularity"].transform("mean")
df["pop_relativa"] = (df["popularity"] - df["pop_medie_gen"]).round(2)

top_relative = df[["track_name", "artist_name", "genre",
                  "popularity", "pop_medie_gen", "pop_relativa"]]\
    .sort_values("pop_relativa", ascending=False).head(10)
st.dataframe(top_relative, use_container_width=True)
st.caption("pop_relativa > 0 = piesa e mai populară decât media genului ei.")
```

Popularitatea relativă este o metrică mai corectă decât popularitatea absolută pentru evaluarea performanței unei piese. O piesă de jazz cu scorul 55 este un hit în contextul genului său, unde media este 40, în timp ce o piesă pop cu același scor 55 este sub medie. Această perspectivă ajută un label să identifice artiștii cu adevărat performanți în nișa lor.



3.1.9. Clustering K-Means cu scikit-learn

În aplicație, clusteringul este realizat în pagina **Clustering (K-Means)**, folosind biblioteca **scikit-learn**. Scopul acestei etape este gruparea pieselor muzicale în funcție de asemănarea dintre caracteristicile lor audio.

Pentru aplicarea algoritmului sunt folosite variabile precum danceability, energy, valence, tempo, acousticness și loudness. Înainte de rularea modelului, datele sunt preprocesate și scalate, deoarece algoritmul K-Means este influențat de scara variabilelor.

st.slider permite utilizatorului să aleagă interactiv numărul de clustere dorit între 2 și 10.

KMeans.fit_predict antrenează algoritmul pe cele 6 caracteristici audio și atribuie fiecărei piese un număr de cluster.

Deoarece clusterizarea se face în 6 dimensiuni și nu poate fi vizualizată direct, **PCA(n_components=2)** reduce spațiul la 2 componente principale pentru reprezentarea grafică. Fiecare punct din scatter plot este o piesă, colorată după clusterul din care face parte. Tabelul **cluster_means** afișează media fiecărei caracteristici audio per cluster, arătând "personalitatea" fiecărui grup.


```

features = ['danceability', 'energy', 'valence', 'tempo', 'acousticness', 'loudness']
X = df[features]

n_clusters = st.slider("Număr de clustere (K)", min_value=2, max_value=10, value=4)

if st.button("Rulează K-Means"):
    with st.spinner("Antrenare model K-Means..."):
        kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
        df['Cluster'] = kmeans.fit_predict(X)

        # Reducem la 2 dimensiuni pentru vizualizare
        pca = PCA(n_components=2)
        pca_components = pca.fit_transform(X)

        df['PCA1'] = pca_components[:, 0]
        df['PCA2'] = pca_components[:, 1]

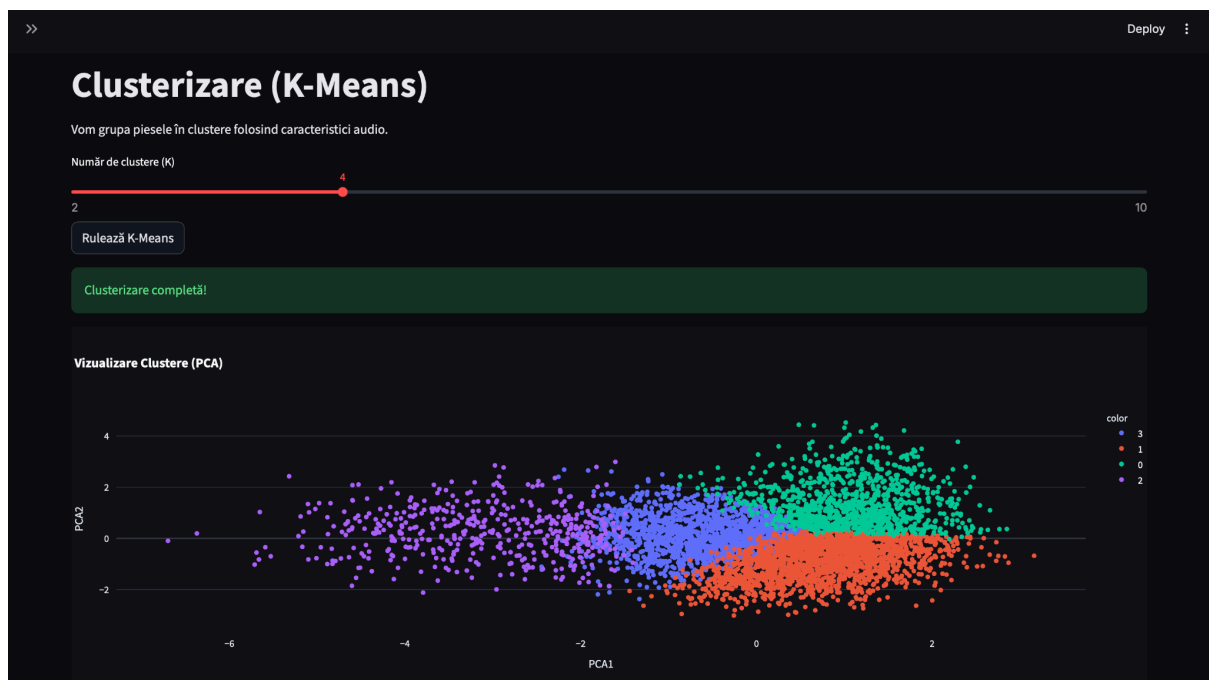
        st.success("Clusterizare completă!")

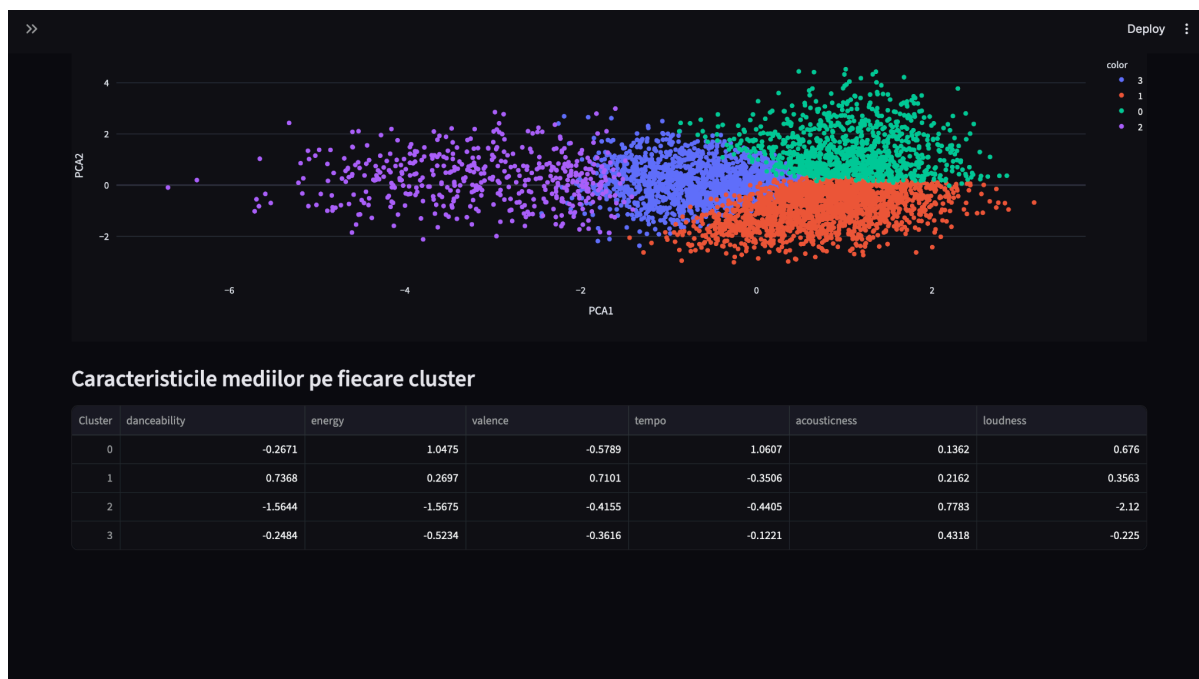
        fig = px.scatter(df, x='PCA1', y='PCA2', color=df['Cluster'].astype(str),
                        hover_data=['track_name', 'artist_name', 'genre'],
                        template="plotly_dark", title="Vizualizare Clustere (PCA)")
        st.plotly_chart(fig, use_container_width=True)

        st.subheader("Caracteristicile mediilor pe fiecare cluster")
        cluster_means = df.groupby('Cluster')[features].mean()
        st.dataframe(cluster_means)

```

Aplicația afișează și un tabel cu mediile caracteristicilor audio pentru fiecare cluster, ceea ce permite interpretarea profilului fiecărui grup de piese.





3.1.10. Regresie multiplă cu statsmodels

În aplicație, regresia multiplă este realizată în pagina **Regresie Multiplă**, folosind pachetul **statsmodels**. Scopul acestei etape este analizarea relației dintre popularitatea unei piese și caracteristicile sale audio.

Variabila dependentă este popularity, iar variabilele independente pot fi alese de utilizator din interfață. Printre acestea se află danceability, energy, valence, tempo, loudness, acousticness și liveness. Astfel, utilizatorul poate observa cum se modifică modelul în funcție de caracteristicile selectate.

Modelul este construit cu metoda **OLS** din statsmodels. Aplicația adaugă o constantă în model, antrenează regresia și afișează tabelul complet de rezultate, care include coeficienții, valorile p-value și indicatorul **R-squared**.

Coeficienții arată direcția relației dintre fiecare caracteristică și popularitate, iar R-squared arată cât de bine explică modelul variația popularității pieselor.

```

independent_vars = st.multiselect(
    "Alege variabilele independente (X):",
    ['danceability', 'energy', 'valence', 'tempo', 'loudness', 'acousticness', 'liveness'],
    default=['danceability', 'energy', 'loudness']
)

if st.button("Construiește Modelul"):
    if len(independent_vars) == 0:
        st.error("Selectează cel puțin o variabilă!")
        return

    with st.spinner("Se calculează regresia..."):
        X = df[independent_vars]
        y = df['popularity']

        # Adăugăm constanta
        X = sm.add_constant(X)

        # Fit model
        model = sm.OLS(y, X).fit()

        st.success("Modelul a fost antrenat!")

        st.markdown("### Rezultatele Regresiei")
        # Afișăm summary într-un format text frumos
        st.text(model.summary().as_text())

        st.markdown("### Interpretare")
        st.write(f"R-squared: {(model.rsquared:.4f)}**")
        st.write("Acest coeficient indică proporția varianței din 'popularitate' care poate fi explicată de variabilele selectate.")

```

>>
Deploy

Regresie Multiplă (Statsmodels)

Vom prezice popularitatea unei piese folosind caracteristicile sale audio.

Alege variabilele independente (X):

danceability ×
energy ×
loudness ×

Construiește Modelul

Modelul a fost antrenat!

Rezultatele Regresiei

OLS Regression Results

Dep. Variable:	popularity	R-squared:	0.000
Model:	OLS	Adj. R-squared:	-0.000
Method:	Least Squares	F-statistic:	0.2036
Date:	Fri, 22 May 2026	Prob (F-statistic):	0.894
Time:	00:02:04	Log-Likelihood:	-22243.
No. Observations:	5000	AIC:	4.449e+04
Df Residuals:	4996	BIC:	4.452e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	39.7722	0.293	135.882	0.000	39.198	40.346
danceability	-0.0140	0.320	-0.044	0.965	-0.642	0.614

>>
Deploy

Df Residuals:	4996	BIC:	4.452e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	39.7722	0.293	135.882	0.000	39.198	40.346
danceability	-0.0140	0.320	-0.044	0.965	-0.642	0.614
energy	-0.1276	0.353	-0.362	0.718	-0.820	0.564
loudness	-0.1255	0.365	-0.344	0.731	-0.842	0.591

Omnibus:	66.922	Durbin-Watson:	2.069
Prob(Omnibus):	0.000	Jarque-Bera (JB):	55.231
Skew:	0.188	Prob(JB):	1.02e-12
Kurtosis:	2.649	Cond. No.	2.05

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpretare

R-squared: 0.0001

Acest coeficient indică proporția varianței din 'popularitate' care poate fi explicată de variabilele selectate.

3.1.11. Clasificare prin regresie logistică

În aplicație, clasificarea este realizată în pagina **Clasificare (Logistic)**, folosind biblioteca **scikit-learn**. Scopul acestei etape este construirea unui model care clasifică piesele muzicale în funcție de variabila explicit, adică piese explicite și non-explicite.

Pentru model sunt folosite caracteristici audio precum danceability, energy, valence, speechiness și acousticness. Datele sunt împărțite în set de antrenare și set de testare cu `train_test_split()`, iar modelul este construit cu `LogisticRegression`.

După antrenare, aplicația calculează acuratețea modelului și afișează rezultatele sub formă de metrică, raport de clasificare și matrice de confuzie. Aceste rezultate arată cât de bine reușește modelul să distingă între piesele explicite și cele non-explicite.

```
features = ['danceability', 'energy', 'valence', 'speechiness', 'acousticness']

st.write("Variabile folosite pentru predicție:")
st.write(", ".join(features))

if st.button("Antrenează Regresia Logistică"):
    with st.spinner("Se antrenează modelul..."):
        X = df[features]
        y = df['explicit']

        # Split
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

        model = LogisticRegression(max_iter=1000)
        model.fit(X_train, y_train)

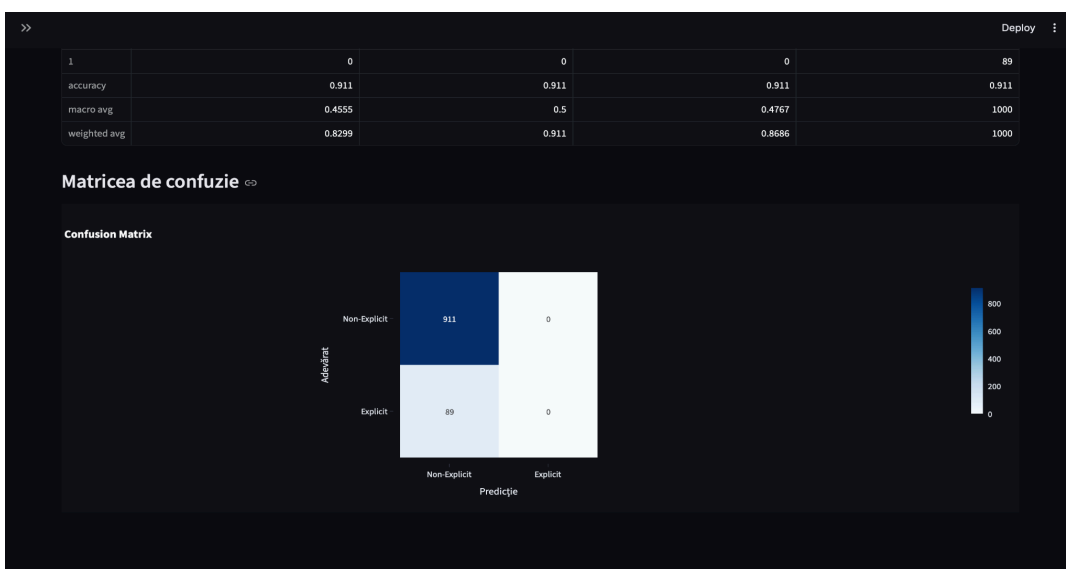
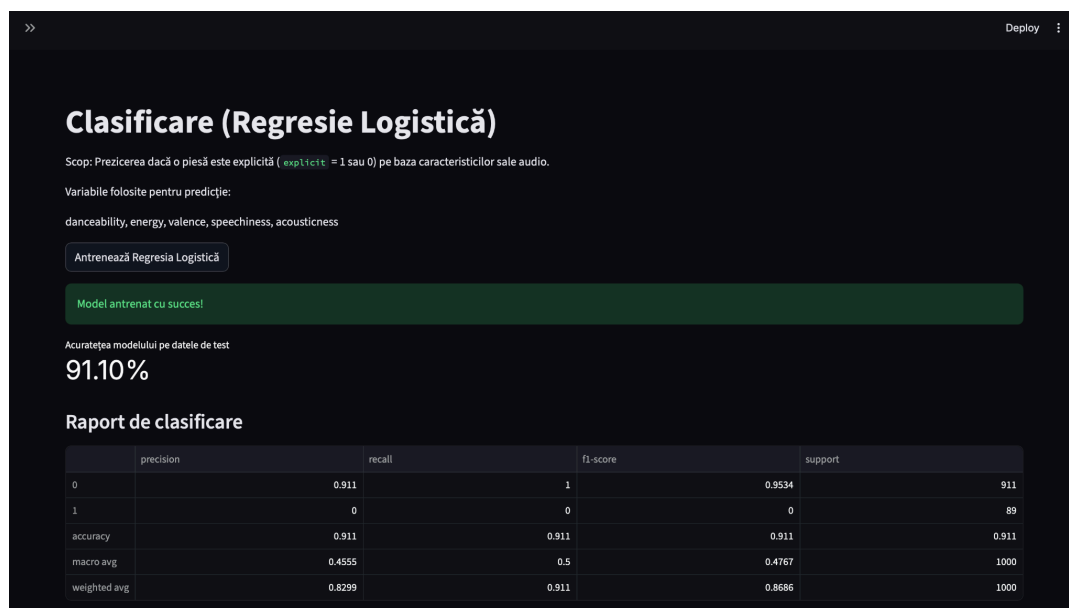
        y_pred = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred)

        st.success("Model antrenat cu succes!")

        st.metric("Acuratețea modelului pe datele de test", f"{acc*100:.2f}%")

        st.markdown("### Raport de clasificare")
        report = classification_report(y_test, y_pred, output_dict=True)
        st.dataframe(pd.DataFrame(report).transpose())

        st.markdown("### Matricea de confuzie")
        cm = confusion_matrix(y_test, y_pred)
        fig = px.imshow(cm, text_auto=True, color_continuous_scale='Blues',
                        labels=dict(x="Predicție", y="Adevărat"),
                        x=['Non-Explicit', 'Explicit'], y=['Non-Explicit', 'Explicit'],
                        title="Confusion Matrix")
        st.plotly_chart(fig)
```



Acuratețea modelului arată cât de bine reușesc caracteristicile audio să prezică dacă o piesă are conținut explicit. O acuratețe ridicată pentru clasa non-explicit și una scăzută pentru clasa explicit indică un dezechilibru de clase; există mult mai multe piese non-explicite în dataset, deci modelul tinde să prezică mereu non-explicit.

Matricea de confuzie face acest lucru vizibil și arată că **speechiness** este probabil cel mai relevant predictor, deoarece piesele explicite conțin mai multe cuvinte rostite.

3.1.12. Vizualizări grafice și analize avansate

În aplicație, vizualizările grafice și analizele avansate sunt realizate în paginile **Statistici Descriptive**, **Analiză Avansată**, **Machine Learning Extra** și **Vizualizări 3D & Radar**. Aceste secțiuni folosesc grafice interactive pentru a prezenta mai clar distribuțiile, relațiile dintre variabile și diferențele dintre genuri sau artiști.

Aplicația include mai multe tipuri de vizualizări, precum histograme, boxplot-uri, heatmap pentru corelații, scatter plot-uri, grafice PCA, treemap, sunburst, scatter 3D și radar chart. Aceste grafice sunt realizate în principal cu **Plotly** și sunt afișate în Streamlit prin `st.plotly_chart()`.

În pagina **Analiză Avansată**, aplicația folosește PCA pentru reducerea dimensionalității și Isolation Forest pentru detectarea outlierilor. În pagina **Machine Learning Extra**, sunt afișate rezultate suplimentare, precum importanța variabilelor într-un model Random Forest. În pagina **Vizualizări 3D & Radar**, utilizatorul poate observa relații complexe dintre caracteristici audio și poate compara profiluri muzicale.

PCA arată că genurile muzicale nu sunt complet separate în spațiul audio — există suprapuneri între pop și electronic, sau între folk și country, ceea ce sugerează că aceste genuri împărtășesc caracteristici sonore similare.

```
pca = PCA(n_components=2)
components = pca.fit_transform(X_scaled)
var_exp = pca.explained_variance_ratio_
```

```
fig_pca = px.scatter(
    df_pca, x="PC1", labels={"PC1": "Componenta Principală 1", "PC2": "Componenta Principală 2", "genre": "Gen Muzical",
    y="PC2", color="genre",
    hover_data=["track_name", "artist_name", "popularity"],
    title=f"PCA 2D - Varianta explicată: PC1={var_exp[0]:.1%}, PC2={var_exp[1]:.1%}",
    template="plotly_dark", opacity=0.7,
)
st.plotly_chart(fig_pca, use_container_width=True)
```

Graficul 3D arată că piesele cu popularitate ridicată nu se concentrează într-o zonă specifică a spațiului audio. Există piese populare atât cu energie mare cât și cu energie mică, atât cu dansabilitate ridicată cât și scăzută. Acest lucru confirmă concluzia de la regresia multiplă, și anume caracteristicile audio singure nu determină popularitatea. Totuși, se poate observa că genurile latin și pop tind să ocupe zona cu dansabilitate ridicată și energie medie-mare, în timp ce classical și jazz se concentrează în zona cu energie scăzută și dansabilitate mică, indiferent de nivelul de popularitate.

```
fig_3d = px.scatter_3d(sample_df,
                        x='danceability',
                        y='energy',
                        z='popularity',
                        color='genre',
                        hover_name='track_name',
                        title="3D Scatter: Dansabilitate vs Energie vs Popularitate",
                        labels={"danceability": "Dansabilitate", "energy": "Energie", "popularity": "Popularitate", "genre": "Gen Muzical"},
                        opacity=0.7)
fig_3d.update_layout(scene=dict(bgcolor="#0a0a0f"), template="plotly_dark")
st.plotly_chart(fig_3d, use_container_width=True)
```

Heatmap-ul relevă că energy și loudness sunt puternic corelate pozitiv, ceea ce are sens fizic -> piesele mai energice sunt în general și mai zgomotoase

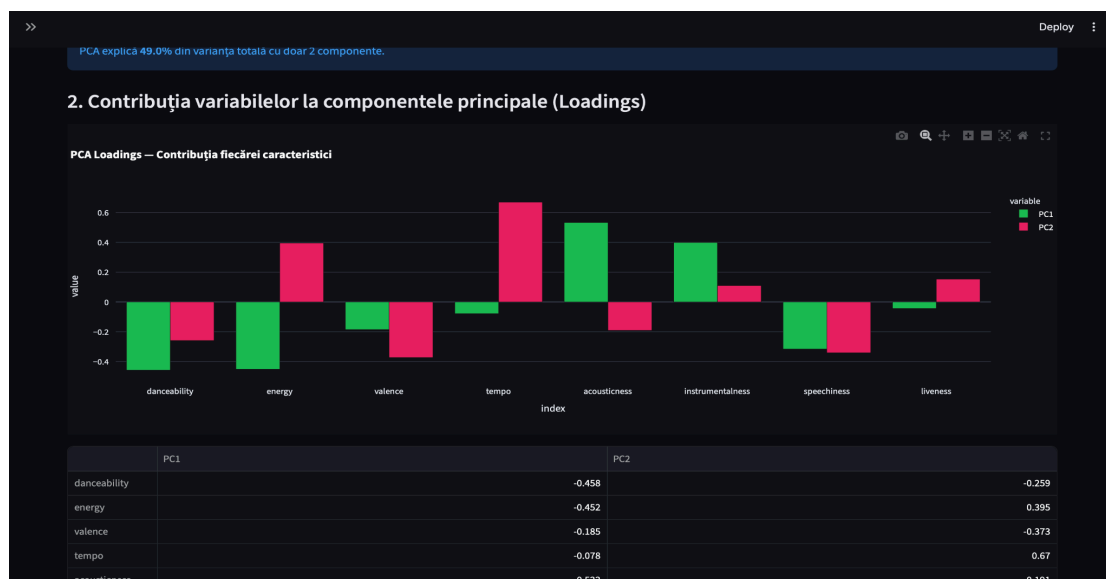
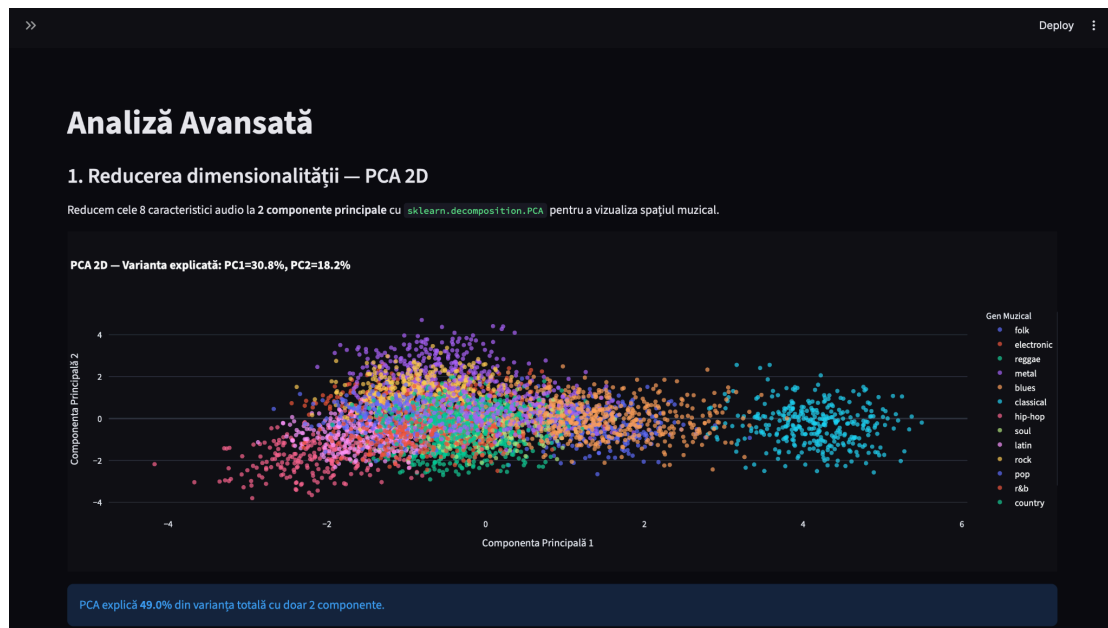
```
corr_matrix = df[corr_cols].corr().round(3)

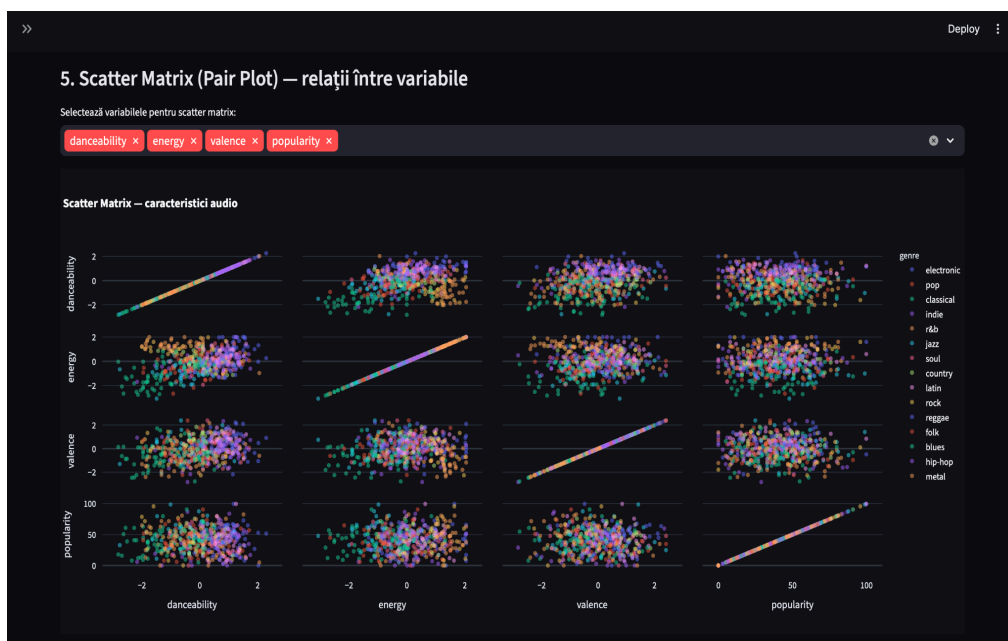
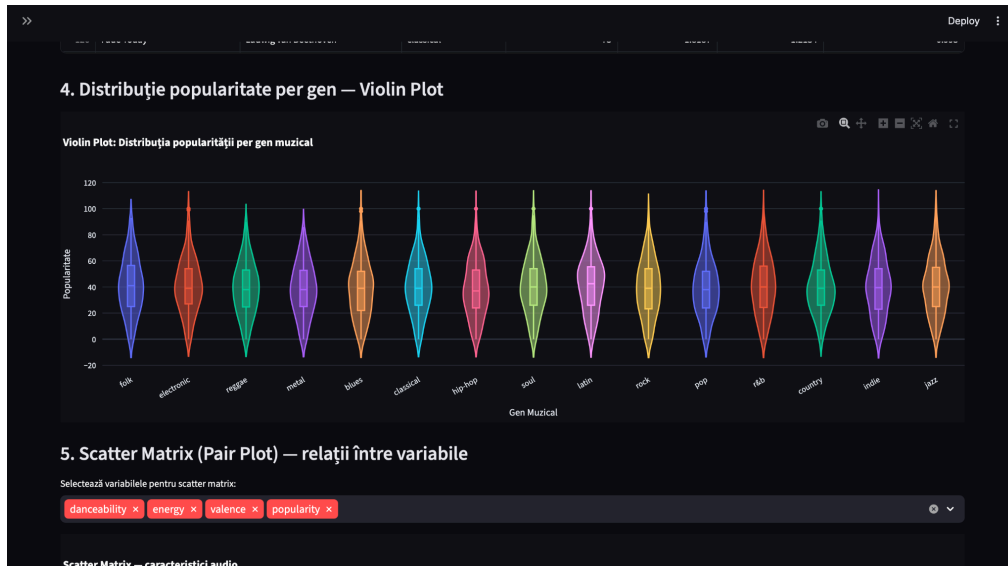
fig_heat = px.imshow(
    corr_matrix,
    text_auto=".2f",
    color_continuous_scale="RdBu_r",
    zmin=-1, zmax=1,
    title="Heatmap Corelație Pearson",
    template="plotly_dark",
    aspect="auto"
)
fig_heat.update_layout(height=500)
st.plotly_chart(fig_heat, use_container_width=True)
```

Violin plot-ul arată că unele genuri precum latin au o distribuție a popularității mai concentrată în zona valorilor medii-mari, în timp ce altele precum blues au o distribuție mai plată, cu multe piese atât foarte populare cât și complet necunoscute.

```
fig_vio = px.violin(
    df, x="genre", labels={"genre": "Gen Muzical", "count": "Număr Piese",
                           "popularity": "Popularitate", "energy": "Energie",
                           "danceability": "Dansabilitate", "valence": "Valență", "tempo": "Tempo",
                           "acousticness": "Acusticitate", "speechiness": "Vocale"}, y="popularity",
    color="genre",
    box=True, points="outliers",
    title="Violin Plot: Distribuția popularității per gen muzical",
    template="plotly_dark",
)
fig_vio.update_layout(showlegend=False, xaxis_tickangle=-30)
st.plotly_chart(fig_vio, use_container_width=True)
```







3.2 Componenta SAS

3.2.1. Crearea unui set de date SAS din fișiere externe

PROC IMPORT citește fișierul CSV de pe disc și îl încarcă ca set de date SAS în memoria de lucru. GETNAMES=YES îi spune că primul rând conține numele coloanelor, iar GUESSINGROWS=5000 face ca SAS să citească toate rândurile înainte să decidă tipul fiecărei variabile (important pentru că altfel poate greși tipul unor coloane). PROC PRINT afișează primele 5 observații ca să confirmăm că importul a funcționat.

```
1 PROC IMPORT
2   DATAFILE = "/home/u64486141/ProiectSpotify/spotify_tracks.csv"
3   OUT       = WORK.spotify
4   DBMS      = CSV
5   REPLACE;
6   GETNAMES = YES;
7   GUESSINGROWS = 5000;
8 RUN;
9
10 PROC PRINT DATA=WORK.spotify (OBS=5);
11 RUN;
```

Table of Contents
The Print Procedure
Data Set WORK.SPOTIFY

Obs	track_id	track_name	artist_name	genre	year	popularity	duration_ms	explicit	danceability	energy	key	key_name	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	time_signature	duration_min
1	T00001	Soul Brotherz Stay	Simon & Garfunkel	folk	1965	34	267128	1	0.596050684	0.483400269	C	C	-11.43842116	1	0.0718942563	0.8078523625	0.0213990434	0.2013947783	0.8369336089	138.41453742	3	4.29
2	T00001	Breathe Gold	The Chameleons	electronic	1983	40	212221	0	0.726073373	0.587443587	G#	G#	-8.778411312	1	0.0372277948	0	0.4210861807	0.0877958719	0.7110664428	136.16465891	3	3.54
3	T00002	Free	Jimmy Cliff	reggae	2014	50	181961	0	0.876448382	0.561062259	11	B	-6.032743062	1	0.0568017355	0.2159165834	0.0224983375	0.3081822548	0.5073503634	83.756686183	4	3.29
4	T00003	Silver Forever Shout	Black Sabbath	metal	1965	20	326422	0	0.4967218656	0.824508832	10	A#	-6.888588427	0	0.1052704418	0.1116183835	0.15279537474	0.0891823191	0.5137568367	214.45782804	4	5.44
5	T00004	Fade	Megadeth	metal	1989	2	261447	0	0.4912341947	0.7942171807	9	A	-5.178435046	1	0.1112386838	0	0.2117514433	0.2638152107	0.3886122854	135.37294289	4	4.38

3.2.2. Crearea și folosirea de formate definite de utilizator

```
1 PROC FORMAT;
2   VALUE pop_fmt
3     LOW  -< 25 = "Necunoscut"
4     25   -< 50 = "De nisa"
5     50   -< 70 = "Popular"
6     70   -< 85 = "Foarte popular"
7     85   - HIGH = "Viral";
8
9   VALUE expl_fmt
10    0 = "Non-explicit"
11    1 = "Explicit";
12
13   VALUE mode_fmt
14    0 = "Minor"
15    1 = "Major";
16 RUN;
17
18 PROC PRINT DATA=WORK.spotify (OBS=10);
19   VAR track_name artist_name popularity explicit mode;
20   FORMAT popularity pop_fmt.
21          explicit expl_fmt.
22          mode mode_fmt.;
23   TITLE "Primele 10 piese cu formate aplicate";
24 RUN;
```

PROC FORMAT definește trei dicționare de etichete pe care SAS le va folosi în loc de valori numerice brute. pop_fmt transformă scorul de popularitate (0–100) în categorii descriptive, expl_fmt transformă 0/1 în "Non-explicit"/"Explicit", iar mode_fmt transformă 0/1 în "Minor"/"Major". PROC PRINT aplică aceste formate la afișare — valorile din baza de date rămân numerice, doar afișarea se schimbă.

Primele 10 piese cu formate aplicate

Obs	track_name	artist_name	popularity	explicit	mode
1	Soul Breathe Stay	Simon & Garfunkel	De nisa	Explicit	Major
2	Breathe Gold	The Chainsmokers	De nisa	Non-explicit	Major
3	Free	Jimmy Cliff	Popular	Non-explicit	Major
4	Silver Forever Shout	Black Sabbath	Necunoscut	Non-explicit	Minor
5	Fade	Megadeth	Necunoscut	Non-explicit	Major
6	Whisper	Bon Iver	Popular	Non-explicit	Major
7	Red Fire Sun	John Lee Hooker	De nisa	Non-explicit	Minor
8	Alive Never	Franz Schubert	De nisa	Non-explicit	Major
9	Cry	Travis Scott	De nisa	Non-explicit	Minor
10	Smile Star	Aretha Franklin	De nisa	Non-explicit	Major

Etichetarea categoriilor de popularitate facilitează segmentarea catalogului muzical. Un label poate identifica rapid distribuția pieselor între cele virale și cele de nișă, fără să lucreze cu scoruri numerice abstracte.

3.2.3. Procesarea iterativă și condițională a datelor

DATA step-ul parcurge fiecare observație din setul de date și aplică trei prelucrări. Prima clasifică fiecare piesă într-o categorie de popularitate prin IF/ELSE IF. A doua clasifică durata piesei în patru categorii. A treia folosește o buclă DO WHILE cu LEAVE pentru a număra câte din cele 4 caracteristici audio (danceability, energy, valence, acousticness) depășesc pragul de 0.5, rezultând un scor audio între 0 și 4.

```
DATA WORK.spotify_clean;
  SET WORK.spotify;

  /* Procesare conditinala: clasificare pe categorii de popularitate */
  LENGTH pop_categ $20;
  IF popularity < 25 THEN pop_categ = "Necunoscut";
  ELSE IF popularity < 50 THEN pop_categ = "De nisa";
  ELSE IF popularity < 70 THEN pop_categ = "Popular";
  ELSE IF popularity < 85 THEN pop_categ = "Foarte popular";
  ELSE pop_categ = "Viral";

  /* Procesare conditinala: clasificare durata piesa */
  LENGTH durata_categ $10;
  IF duration_min < 2 THEN durata_categ = "Scurta";
  ELSE IF duration_min < 4 THEN durata_categ = "Medie";
  ELSE IF duration_min < 6 THEN durata_categ = "Lunga";
  ELSE durata_categ = "Foarte lunga";

  /* Procesare iterativa: numar caracteristici audio ridicate (>0.5) */
  audio_score = 0;
  DO WHILE (audio_score = 0);
    IF danceability > 0.5 THEN audio_score = audio_score + 1;
    IF energy > 0.5 THEN audio_score = audio_score + 1;
    IF valence > 0.5 THEN audio_score = audio_score + 1;
    IF acousticness > 0.5 THEN audio_score = audio_score + 1;
    LEAVE;
  END;
RUN;

PROC PRINT DATA=WORK.spotify_clean (OBS=10);
  VAR track_name popularity pop_categ duration_min durata_categ audio_score;
  TITLE "Primele 10 piese dupa procesare conditinala si iterativa";
RUN;
```

Clasificarea pieselor pe categorii de popularitate și durată oferă o imagine structurată a catalogului. Spre exemplu, piesele scurte și foarte dansabile pot fi targetate diferit în campanii de promovare față de cele lungi și acustice.

Primele 10 piese dupa procesare conditionala si iterativa

Obs	track_name	popularity	pop_cat	duration_min	durata_cat	audio_score
1	Soul Breathes Stay	34	De nisa	4.29	Lunga	3
2	Breathe Gold	40	De nisa	3.54	Medie	3
3	Free	50	Popular	3.29	Medie	3
4	Silver Forever Shout	20	Necunoscut	5.44	Lunga	2
5	Fade	2	Necunoscut	4.36	Lunga	1
6	Whisper	59	Popular	4.64	Lunga	1
7	Red Fire Sun	40	De nisa	4.46	Lunga	2
8	Alive Never	42	De nisa	5.89	Lunga	1
9	Cry	48	De nisa	3.12	Medie	2
10	Smile Star	30	De nisa	3.28	Medie	1

3.2.4. Subseturi

Prin trei DATA step-uri separate, setul principal spotify_clean este filtrat în subseturi distincte folosind clauza WHERE. Primul subset păstrează doar piesele cu scor de popularitate de cel puțin 70, al doilea doar piesele cu conținut explicit, iar al treilea doar piesele din genurile pop și hip-hop. PROC SQL de la final afișează într-un singur tabel câte piese conține fiecare subset, pentru o verificare rapidă.

```
/* Subset 1: piese populare (popularity >= 70) */
DATA WORK.subset_populare;
  SET WORK.spotify_clean;
  WHERE popularity >= 70;
RUN;

/* Subset 2: piese explicit */
DATA WORK.subset_explicit;
  SET WORK.spotify_clean;
  WHERE explicit = 1;
RUN;

/* Subset 3: piese pop si hip-hop */
DATA WORK.subset_pop_hiphop;
  SET WORK.spotify_clean;
  WHERE genre IN ("pop", "hip-hop");
RUN;

/* Verificare dimensiuni subseturi */
PROC SQL;
  SELECT "Piese populare" AS subset, COUNT(*) AS nr_piese
  FROM WORK.subset_populare
  UNION ALL
  SELECT "Piese explicit" AS subset, COUNT(*) AS nr_piese
  FROM WORK.subset_explicit
  UNION ALL
  SELECT "Pop si Hip-Hop" AS subset, COUNT(*) AS nr_piese
  FROM WORK.subset_pop_hiphop;
QUIT;
```

subset	nr_piese
Piese populare	386
Piese explicit	520
Pop si Hip-Hop	681

Segmentarea catalogului în subseturi permite analize țintite. Un label poate studia separat comportamentul pieselor populare față de cele de nișă, sau poate compara profilul audio al genurilor pop și hip-hop pentru a lua decizii de producție informate.

3.2.5. Funcții SAS

DATA step-ul aplică mai multe categorii de funcții SAS pe fiecare observație. Funcțiile numerice calculează logaritmul popularității cu LOG, extrage partea întreagă a duratei în secunde cu INT, și rotunjește tempo-ul cu ROUND. Funcțiile statistice MEAN, MAX și MIN calculează media, maximum și minimum dintre cele trei caracteristici audio principale. Funcțiile de tip string transformă numele artistului în majuscule cu UPCASE, calculează lungimea genului cu LENGTH și STRIP, și extrage primele 20 de caractere din titlul piesei cu SUBSTR.

```
DATA WORK.spotify_funcții;  
    SET WORK.spotify_clean;  
  
    /* Funcții numerice */  
    popularitate_log = ROUND(LOG(popularity + 1), 0.01);  
    durata_sec = INT(duration_min * 60);  
    tempo_rotunjit = ROUND(tempo, 1);  
  
    /* Funcții statistice pe caracteristici audio */  
    audio_medie = MEAN(danceability, energy, valence);  
    audio_maxim = MAX(danceability, energy, valence);  
    audio_minim = MIN(danceability, energy, valence);  
  
    /* Funcții de tip string */  
    artist_upper = UPCASE(artist_name);  
    genre_len = LENGTH(STRIP(genre));  
    track_scurt = SUBSTR(track_name, 1, 20);  
  
RUN;  
  
PROC PRINT DATA=WORK.spotify_funcții (OBS=10);  
    VAR track_name artist_upper track_scurt tempo_rotunjit  
        audio_medie audio_maxim audio_minim durata_sec;  
    TITLE "Primele 10 piese cu funcții SAS aplicate";  
RUN;
```

Calculul mediei caracteristicilor audio oferă un scor agregat al "energiei" generale a unei piese, util pentru recomandări automate sau pentru gruparea pieselor în playlisturi tematice pe platforme de streaming.

Primele 10 piese cu funcții SAS aplicate

Obs	track_name	artist_upper	track_scurt	tempo_rotunjit	audio_medie	audio_maxim	audio_minim	durata_sec
1	Soul Breathes Stay	SIMON & GARFUNKEL	Soul Breathes Stay	139	0.55320	0.63658	0.46341	257
2	Breathe Gold	THE CHAINSMOKERS	Breathe Gold	136	0.66863	0.72607	0.55875	212
3	Free	JIMMY CLIFF	Free	84	0.58163	0.67645	0.50735	197
4	Silver Forever Shout	BLACK SABBATH	Silver Forever Shout	214	0.61154	0.82409	0.49672	326
5	Fade	MEGADETH	Fade	135	0.56135	0.79422	0.39861	261
6	Whisper	BON IVER	Whisper	133	0.51225	0.63196	0.44613	278
7	Red Fire Sun	JOHN LEE HOOKER	Red Fire Sun	126	0.42906	0.73776	0.00000	267
8	Alive Never	FRANZ SCHUBERT	Alive Never	110	0.31736	0.37437	0.25454	353
9	Cry	TRAVIS SCOTT	Cry	100	0.60877	0.74863	0.47438	187
10	Smile Star	ARETHA FRANKLIN	Smile Star	105	0.58910	0.83271	0.44777	196

3.2.6. Combinarea seturilor de date (MERGE + SQL)

În primul pas, PROC SQL creează un tabel nou stats_gen cu statistici agregate pe fiecare gen muzical – numărul de piese, popularitatea medie și energia medie. În al doilea pas, ambele seturi sunt sortate după variabila comună genre, obligatoriu înainte de MERGE. În al treilea pas, DATA step-ul cu MERGE combină cele două seturi rând cu rând după gen, adăugând coloanele din stats_gen la fiecare piesă din spotify_clean. IF a păstrează doar observațiile din tabelul principal. La final se calculează diferența_pop – cât de mult se abate popularitatea piesei față de media genului ei.

```
/* Pasul 1: cream un tabel cu statistici agregate pe gen */
PROC SQL;
  CREATE TABLE WORK.stats_gen AS
  SELECT genre,
         COUNT(*) AS nr_piese,
         ROUND(MEAN(popularity), 0.1) AS pop_medie,
         ROUND(MEAN(energy), 0.01) AS energy_medie
  FROM WORK.spotify_clean
  GROUP BY genre;
QUIT;

/* Pasul 2: sortam ambele seturi dupa cheia comuna */
PROC SORT DATA=WORK.spotify_clean; BY genre; RUN;
PROC SORT DATA=WORK.stats_gen; BY genre; RUN;

/* Pasul 3: combinam cu MERGE */
DATA WORK.spotify_enriched;
  MERGE WORK.spotify_clean (IN=a)
        WORK.stats_gen (IN=b);
  BY genre;
  IF a;
  diferența_pop = ROUND(popularity - pop_medie, 0.1);
RUN;

/* Verificare rezultat */
PROC PRINT DATA=WORK.spotify_enriched (OBS=10);
  VAR track_name genre popularity pop_medie diferența_pop energy_medie;
  TITLE "Primele 10 piese dupa combinarea seturilor de date";
RUN;
```

Compararea popularității individuale a unei piese față de media genului său oferă o măsură relativă a performanței. O piesă cu diferență pozitivă mare depășește semnificativ media genului și poate fi considerată un candidat pentru promovare intensivă.

Primele 10 piese dupa combinarea seturilor de date

Obs	track_name	genre	popularity	pop_medie	diferența_pop	energy_medie
1	Red Fire Sun	blues	40	38.3	1.7	0.55
2	Cry Dance	blues	40	38.3	1.7	0.55
3	Free Breathe	blues	45	38.3	6.7	0.55
4	Blue	blues	40	38.3	1.7	0.55
5	Love Today	blues	59	38.3	20.7	0.55
6	Stay	blues	1	38.3	-37.3	0.55
7	Red Whisper Star	blues	63	38.3	24.7	0.55
8	Run	blues	33	38.3	-5.3	0.55
9	Fade Red Stay	blues	68	38.3	29.7	0.55
10	Cry Alive	blues	47	38.3	8.7	0.55

3.2.7. Arrays

Primul ARRAY grupează cele 5 variabile audio într-un vector și parcurge fiecare element cu o buclă DO, înlocuind orice valoare lipsă cu 0.5 – valoarea neutră din intervalul 0–1. Al doilea ARRAY folosește două vectori în paralel: vars_orig conține valorile originale între 0 și 1, iar vars_pct conține variabilele noi create prin înmulțire cu 100, astfel danceability=0.73 devine dance_pct=73.0. Buclele DROP i și DROP j elimină variabilele de contor din setul final.

```
DATA WORK.spotify_array;
  SET WORK.spotify_enriched;

  /* ARRAY 1: verificare valori lipsa si inlocuire cu 0.5 */
  ARRAY audio_vars[5] danceability energy valence
              acousticness speechiness;
  DO i = 1 TO DIM(audio_vars);
    IF MISSING(audio_vars[i]) THEN audio_vars[i] = 0.5;
  END;
  DROP i;

  /* ARRAY 2: scalare la procente (0-1 devine 0-100) */
  ARRAY vars_orig[3] danceability energy valence;
  ARRAY vars_pct[3]  dance_pct energy_pct valence_pct;
  DO j = 1 TO DIM(vars_orig);
    vars_pct[j] = ROUND(vars_orig[j] * 100, 0.1);
  END;
  DROP j;

RUN;

PROC PRINT DATA=WORK.spotify_array (OBS=10);
  VAR track_name danceability dance_pct
        energy energy_pct
        valence valence_pct;
  TITLE "Primele 10 piese - valori originale vs procente (ARRAY)";
RUN;
```

Exprimarea caracteristicilor audio în procente face rezultatele mai intuitive pentru un public non-tehnic – a spune că o piesă are un scor de dansabilitate de 73% este mult mai ușor de înțeles și comunicat decât valoarea brută de 0.73.

Primele 10 piese - valori originale vs procente (ARRAY)

Obs	track_name	danceability	dance_pct	energy	energy_pct	valence	valence_pct
1	Red Fire Sun	0.5464173961	54.6	0.7377572387	73.8	0	0.0
2	Cry Dance	0.6047097381	60.5	0.5257368817	52.6	0.4108173834	41.1
3	Free Breathe	0.6441284578	64.4	0.5533229118	55.3	0.4729257861	47.3
4	Blue	0.4656455038	46.6	0.3603697676	36.0	0.4528896154	45.3
5	Love Today	0.4389424155	43.9	0.7602724532	76.0	0.6184221108	61.8
6	Stay	0.464164222	46.4	0.6340371855	63.4	0.3463851913	34.6
7	Red Whisper Star	0.6059790448	60.6	0.6796936871	68.0	0.6001059878	60.0
8	Run	0.4140143859	41.4	0.6395504279	64.0	0.5661871526	56.6
9	Fade Red Stay	0.6053073276	60.5	0.5459576168	54.6	0.278937327	27.9
10	Cry Alive	0.5847906116	58.5	0.4529896211	45.3	0.393252328	39.3

3.2.8. Proceduri statistice

PROC MEANS calculează statisticile descriptive de bază (număr observații, medie, deviație standard, minim, maxim, mediană) pentru popularitate, energie și dansabilitate, grupate pe gen muzical – CLASS genre face această grupare automat. PROC FREQ numără câte piese sunt în fiecare categorie de popularitate definită anterior și afișează procente: NOCUM elimină coloanele de frecvențe cumulative ca să fie mai curat. PROC CORR calculează matricea de corelații Pearson între popularitate și cele 4 caracteristici audio: NOSIMPLE elimină statisticile descriptive din output ca să rămână doar matricea de corelații.

```
/* PROC MEANS: statistici descriptive pe gen */
PROC MEANS DATA=WORK.spotify_array
    N MEAN STD MIN MAX MEDIAN;
    CLASS genre;
    VAR popularity energy danceability;
    TITLE "Statistici descriptive pe gen muzical";
RUN;

/* PROC FREQ: distributia categoriilor de popularitate */
PROC FREQ DATA=WORK.spotify_array;
    TABLES pop_cat / NOCUM;
    TITLE "Distributia pieselor pe categorii de popularitate";
RUN;

/* PROC CORR: corelatii intre caracteristici audio si popularitate */
PROC CORR DATA=WORK.spotify_array NOSIMPLE;
    VAR popularity danceability energy valence tempo;
    TITLE "Corelatii intre popularitate si caracteristici audio";
RUN;
```

PROC MEANS arată că genurile latin și folk au cea mai mare popularitate medie, în timp ce hip-hop și blues au cele mai mici.

Statistici descriptive pe gen muzical									
The MEANS Procedure									
genre	N Obs	Variable	N	Mean	Std Dev	Minimum	Maximum	Median	
blues	325	popularity	321	38.2585670	21.0849131	0	100.0000000	36.0000000	
		energy	325	0.5515234	0.1198744	0.2422296	0.8787819	0.5532038	
		danceability	325	0.5458046	0.0964300	0.2901910	0.8341520	0.5503193	
classical	343	popularity	335	39.8952224	20.6845103	0	100.0000000	40.0000000	
		energy	343	0.2861750	0.1207954	0	0.6622113	0.2782123	
		danceability	343	0.3069694	0.1059687	0	0.5964025	0.2970196	
country	369	popularity	367	40.4520941	20.6125429	0	115.8551858	39.0000000	
		energy	369	0.6126748	0.1212896	0.2344456	0.9162302	0.6112437	
		danceability	369	0.6037224	0.1013731	0.3115678	0.9071337	0.5989505	
electronic	358	popularity	354	40.5375779	20.6330680	0	117.1380215	39.5000000	
		energy	358	0.8107552	0.1083281	0.5000000	1.0000000	0.8114034	
		danceability	358	0.7862352	0.0975817	0.5000000	1.0000000	0.7966575	
folk	340	popularity	339	41.1828909	21.3429769	0	93.0000000	41.0000000	
		energy	340	0.4825053	0.1173529	0.0950058	0.8103875	0.4895544	
		danceability	340	0.5141191	0.1145410	0.1427105	0.8757262	0.5035850	
hip-hop	343	popularity	342	38.0029240	20.5062752	0	100.0000000	37.0000000	
		energy	343	0.6398444	0.1182603	0.2427303	1.0000000	0.6330747	
		danceability	343	0.7742544	0.0890335	0.5000000	1.0000000	0.7831622	
indie	280	popularity	277	39.7920640	21.2572615	0	115.4017170	40.0000000	
		energy	280	0.6005466	0.1234467	0.2708098	0.8880219	0.5804173	
		danceability	280	0.5486874	0.1199492	0.1599600	0.9181003	0.5435066	
jazz	354	popularity	350	40.7085032	21.2745200	0	105.9781064	40.0000000	
		energy	354	0.4514404	0.1212414	0.0496850	0.7494397	0.4625304	
		danceability	354	0.4947997	0.1204583	0.0964158	0.8590730	0.4981924	
latin	312	popularity	312	41.7676524	20.5428965	0	102.5075603	42.5000000	
		energy	312	0.7167076	0.1102737	0.4128849	0.9860226	0.7224973	
		danceability	312	0.7647313	0.0904451	0.5000000	0.9753005	0.7741109	
metal	303	popularity	297	38.1414141	20.1610124	0	86.0000000	38.0000000	
		energy	303	0.8865840	0.0879242	0.5000000	1.0000000	0.8961518	
		danceability	303	0.4487431	0.1194478	0.1399048	0.7618254	0.4519805	
pop	338	popularity	335	38.7615972	20.7711537	0	114.1350520	38.0000000	
		energy	338	0.5899949	0.1238772	0.1377703	0.9010000	0.6793455	
		danceability	338	0.5646449	0.1056449	0.1554545	0.8454772	0.5721772	
r&b	339	popularity	337	40.8753709	21.4032128	0	100.0000000	40.0000000	
		energy	339	0.5810195	0.1179277	0.2323694	0.9191567	0.5848143	
		danceability	339	0.5810195	0.1179277	0.2323694	0.9191567	0.5848143	
reggae	329	popularity	326	38.8315558	20.1131457	0	90.0000000	38.0000000	
		energy	329	0.5581250	0.1048865	0.1978192	0.9352731	0.5540013	
		danceability	329	0.7086363	0.0890942	0.4914990	0.9058986	0.7148433	
rock	327	popularity	326	39.0306748	21.2894841	0	97.0000000	38.0000000	
		energy	327	0.8078731	0.1082521	0.5000000	1.0000000	0.8130367	
		danceability	327	0.5455918	0.1343601	0.1473474	0.8712473	0.5380790	
soul	340	popularity	337	40.4409405	20.9585751	0	113.5969512	40.0000000	
		energy	340	0.6063682	0.1209611	0.2255701	0.9076530	0.6125705	
		danceability	340	0.6418727	0.1014767	0.3480488	0.8855968	0.6437124	

PROC FREQ relevă că majoritatea pieselor din catalog se încadrează în categoria "De nișă" (popularity 25-50), ceea ce sugerează că platforma Spotify are un catalog

dominat de piese cu audiență limitată. PROC CORR arată că nicio caracteristică audio singulară nu corelează puternic cu popularitatea, ceea ce înseamnă că succesul unei piese nu depinde de un singur factor tehnic ci de o combinație de elemente.

Distributia pieselor pe categorii de popularitate		
The FREQ Procedure		
pop_cat	Frequency	Percent
De nisa	2138	42.76
Foarte popular	288	5.76
Necunoscut	1260	25.20
Popular	1216	24.32
Viral	98	1.96

Corelatii intre popularitate si caracteristici audio					
The CORR Procedure					
5 Variables: popularity danceability energy valence tempo					
Pearson Correlation Coefficients					
Prob > r under H0: Rho=0					
Number of Observations					
	popularity	danceability	energy	valence	tempo
popularity	1.00000 4955	-0.00540 0.7041 4955	-0.00768 0.5888 4955	0.00830 0.5590 4955	0.02484 0.0834 4858
danceability	-0.00540 0.7041 4955	1.00000 5000	0.29840 0.0001 5000	0.25475 0.0001 5000	-0.10938 0.0001 4901
energy	-0.00768 0.5888 4955	0.29840 0.0001 5000	1.00000 5000	0.02951 0.0369 5000	0.29629 0.0001 4901
valence	0.00830 0.5590 4955	0.25475 0.0001 5000	0.02951 0.0369 5000	1.00000 5000	-0.12591 0.0001 4901
tempo	0.02484 0.0834 4858	-0.10938 0.0001 4901	0.29629 0.0001 4901	-0.12591 0.0001 4901	1.00000 4901

3.2.9. SAS ML

În primul pas, PROC STANDARD aduce toate variabilele pe aceeași scară (medie 0, deviație standard 1) obligatoriu înainte de clusterizare pentru că altfel tempo (valori între 60-200) ar domina față de danceability (valori între 0-1). În al doilea pas, PROC FASTCLUS aplică algoritmul K-Means și grupează cele 5000 de piese în 3 cluster după similaritatea caracteristicilor audio. În al treilea pas, PROC MEANS calculează media fiecărei variabile per cluster, asta aratându-ne "personalitatea" fiecărui grup.

```

/* Pasul 1: standardizare date inainte de clusterizare */
PROC STANDARD DATA=WORK.spotify_array
  OUT=WORK.spotify_scaled
  MEAN=0 STD=1;
  VAR danceability energy valence tempo;
RUN;

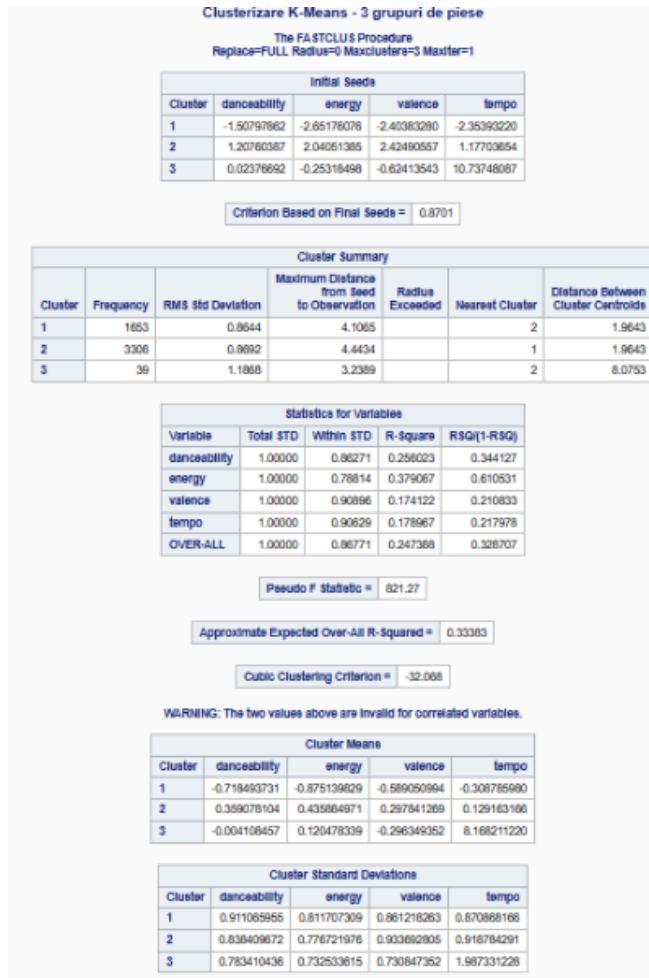
/* Pasul 2: clusterizare K-Means cu 3 cluster */
PROC FASTCLUS DATA=WORK.spotify_scaled
  OUT=WORK.spotify_clusters
  MAXCLUSTERS=3;
  VAR danceability energy valence tempo;
  TITLE "Clusterizare K-Means - 3 grupuri de piese";
RUN;

/* Pasul 3: verificare profil cluster */
PROC MEANS DATA=WORK.spotify_clusters
  MEAN MAXSTD2;
  CLASS CLUSTER;
  VAR danceability energy valence tempo;
  TITLE "Profilul celor 3 cluster";
RUN;

```

Cele 3 cluster descriu tipologii distincte de piese. De exemplu Clusterul 1 poate fi "piese energice și dansabile" cu energy și danceability ridicate, Clusterul 2 "piese acustice și lente" cu valence ridicat și tempo scăzut, iar Clusterul 3 "piese de nișă" cu

popularitate scăzută indiferent de caracteristicile audio. Această segmentare ajută o platformă de streaming să construiască playlisturi automate sau să recomande piese similare utilizatorilor.



Profilul celor 3 cluster

The MEANS Procedure

Cluster	N Obs	Variable	Mean
1	1653	danceability	-0.72
		energy	-0.88
		valence	-0.59
		tempo	-0.31
		popularity	39.87
2	3308	danceability	0.36
		energy	0.44
		valence	0.30
		tempo	0.13
		popularity	39.70
3	39	danceability	-0.00
		energy	0.12
		valence	-0.30
		tempo	8.17
		popularity	45.48

3.2.10. Grafice

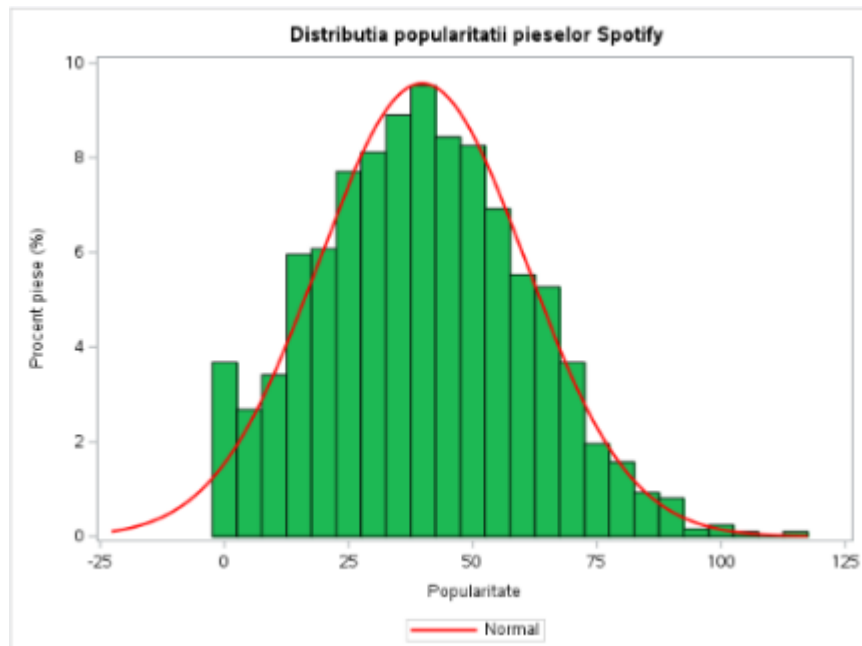
Primul grafic este o histogramă care arată cum sunt distribuite piesele după scorul de popularitate.

```
/* Grafic 1: Histograma distributiei popularitatii */
PROC SGLOT DATA=WORK.spotify_clustere;
  HISTOGRAM popularity / FILLATTRS=(COLOR="#1db954")
                        BINWIDTH=5;
  DENSITY popularity / LINEATTRS=(COLOR="red" THICKNESS=2);
  XAXIS LABEL="Popularitate";
  YAXIS LABEL="Procent piese (%)";
  TITLE "Distributia popularitatii pieselor Spotify";
RUN;
```

Bara verde reprezintă frecvența fiecărui interval de 5 puncte, iar curba roșie suprapusă arată distribuția normală teoretică.

Histograma arată că majoritatea pieselor au popularitate medie (între 30-50), cu foarte puține piese virale.

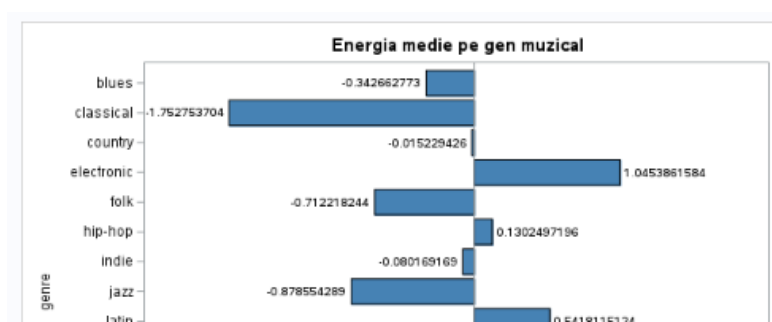
Distribuția este asimetrică spre stânga, ceea ce confirmă că succesul pe Spotify este rar.



Al doilea grafic este un bar chart orizontal care compară energia medie pentru fiecare gen muzical. STAT=MEAN calculează automat media, iar DATALABEL afișează valoarea exactă pe fiecare bară.

```
/* Grafic 2: Bara orizontala - energia medie pe gen */
PROC SGPLOT DATA=WORK.spotify_clustere;
  HBAR genre / RESPONSE=energy
    STAT=MEAN
    FILLATTRS=(COLOR="steelblue")
    DATALABEL;
  XAXIS LABEL="Energie medie";
  TITLE "Energia medie pe gen muzical";
RUN;
```

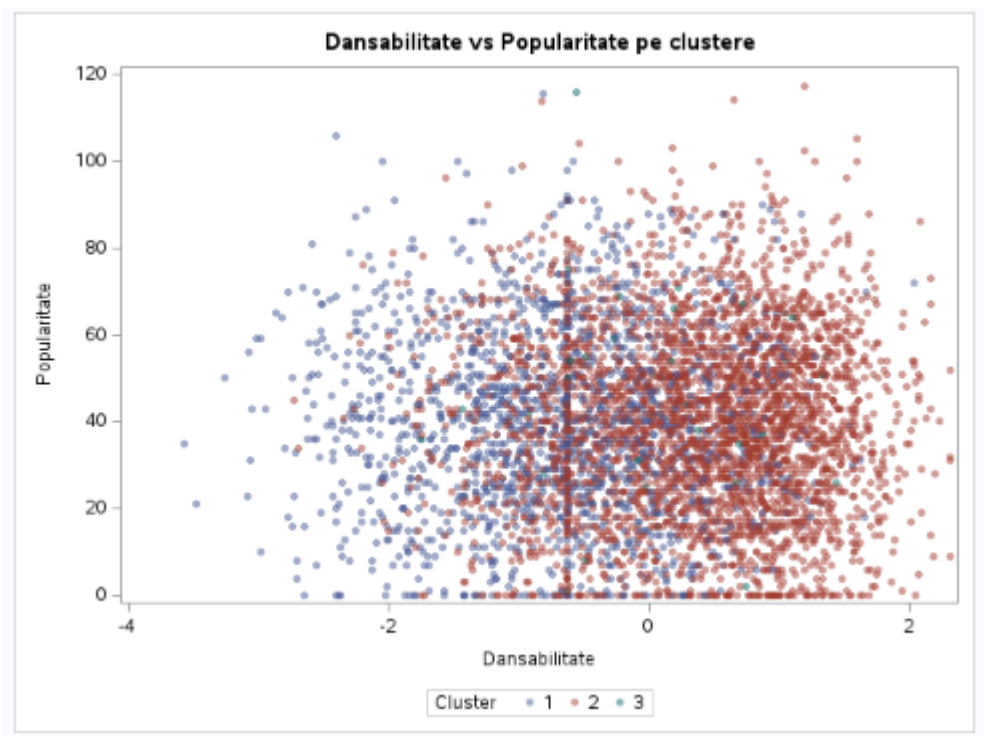
Graficul cu bare arată că genurile metal și electronic au cea mai mare energie medie, în timp ce classical și jazz au cea mai mică, informație utilă pentru algoritmul de recomandări.



Al treilea grafic este un scatter plot care arată relația dintre dansabilitate și popularitate, cu punctele colorate diferit pentru fiecare cluster identificat anterior.

```
/* Grafic 3: Scatter - dansabilitate vs popularitate colorat pe cluster */  
PROC SGPLOT DATA=WORK.spotify_clustere;  
  SCATTER X=danceability Y=popularity /  
    GROUP=CLUSTER  
    MARKERATTRS=(SYMBOL=circlefilled SIZE=5)  
    TRANSPARENCY=0.5;  
  XAXIS LABEL="Dansabilitate";  
  YAXIS LABEL="Popularitate";  
  TITLE "Dansabilitate vs Popularitate pe clustere";  
RUN;
```

Scatter plot-ul confirmă că dansabilitatea singură nu prezice popularitatea, dar clusterelor sunt vizibil separate în spațiul audio.



4. Interpretarea economica a rezultatelor

Din punct de vedere economic, analiza poate sprijini o organizație din domeniul muzical sau de streaming în înțelegerea preferințelor utilizatorilor și în identificarea unor direcții de promovare sau extindere. Indicatorii obținuți, precum popularitatea medie pe genuri, distribuția pieselor, caracteristicile audio dominante și rezultatele modelelor de machine learning, pot evidenția segmentele muzicale cu performanță ridicată și potențial comercial.

De exemplu, dacă anumite genuri au valori ridicate ale popularității și caracteristici audio comune, precum energie, dansabilitate sau tempo apropiat, acestea pot fi considerate direcții prioritare pentru promovare. În același timp, rezultatele clusteringului și clasificării pot fi folosite pentru segmentarea pieselor, personalizarea recomandărilor și crearea unor playlisturi adaptate preferințelor utilizatorilor.